WEB DESIGN

HTML • CSS • JavaScript

Web Design Course By Instructor Eric BYIRINGIRO(0782427392)

Course Overview

This self-paced course is designed to provide the necessary skills and training for an entry-level position in the field of Web design. The student learns to develop and maintain Web sites for a corporation of one's small business. The class focuses on Web page planning, basic design, layout and construction, and setup and maintenance of a web site.

HTML

CSS

JavaScript

PHP

MySQL

Learning Outcomes

- To understand Web and its relationship with the Internet
- To know who is a Designer, what he/she has to do and different specializations of Web designers
- To describe different tools used in order to design a professional Web site
- To write tags/codes of HTML, CSS and JavaScript, three languages mainly discussed in this lesson
- To develop and to maintain a Web site for a small organization
- To be able to customize the existing web pages and reuse them in our projects

Course Contents

- Introducing Web Design
- Hypertext Markup Language (HTML)
  - HTML Text Layout Tags
  - Understanding Hyperlinks
  - Adding Images to Web Pages
  - HTML Tables
  - HTML Forms
- Cascading Style Sheets (CSS)
  - Formatting Fonts and Text with CSS
  - CSS Colors, Padding, Margins & Borders
- Introduction to JavaScript

The Process of Interacting with a Website

1 User Input

User enters URL or clicks link; browser prepares request

2 HTTP Request

Browser sends HTTP/HTTPS request — may include form data or API call

3 Server Processing

Server executes back-end code (PHP, Node.js, Python); queries database

4 Server Response

Server sends back HTML, CSS, JS, or JSON data

5 Browser Rendering

Browser converts raw code into the visual, interactive webpage you see on your screen

6 Ongoing Interaction

User continues interacting; each action may repeat the cycle

Front-End vs Back-End Development

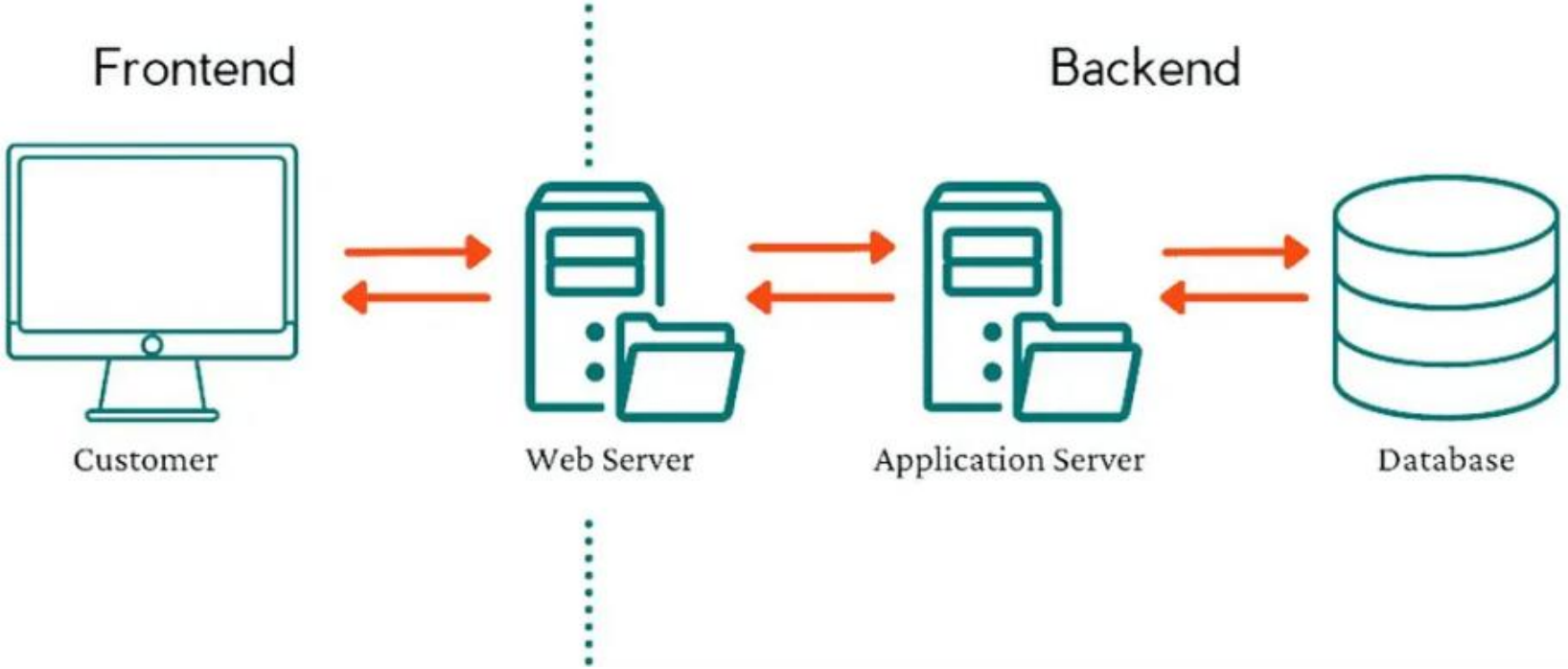
Front-End

- Visual part users interact with
- Languages: HTML, CSS, JavaScript
- Framework: Bootstrap
  - Bootstrap is a frontend toolkit consisting of pre-styled CSS elements and utility classes used for layout and design.
- Tools: Figma, Adobe XD, Sketch
  - Figma – A tool for UI/UX design, prototyping, and vector graphics.
  - Adobe XD – A vector-based application for designing and prototyping user experiences.
  - Sketch – A macOS-based vector graphics editor focused on UI/UX design.
- Focus: UI/UX, responsive design

Back-End

- Part Responsible for executing operations, managing data, and communicating with databases or APIs
- Server, database & logic
- Languages: Node.js, Python, PHP, Java
- Databases: MySQL, PostgreSQL, MongoDB
- Tasks: APIs, authentication, data management
- Handles requests from the front-end
- Full-stack: expertise in both sides

How to Connect Frontend and Backend Applications



HTML

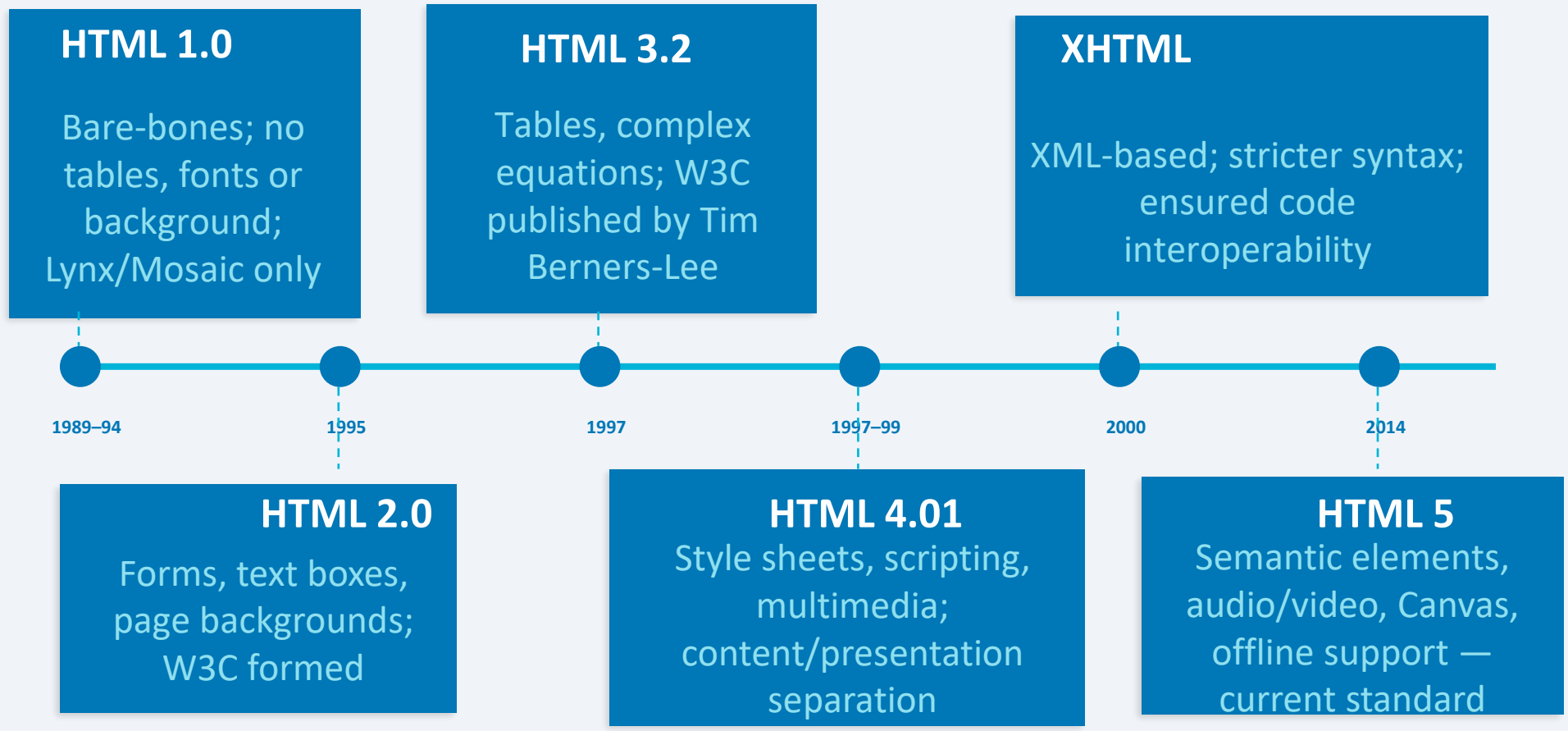
1

Hyper Text Markup Language — The Structure of the Web

What is HTML?

- HTML = HyperText Markup Language — the standard language for creating web pages
- Technically a markup language, not a programming language — it describes structure and content
- HyperText: allows navigation between pages via clickable hyperlinks
- Markup: HTML tags instruct the browser how to display content
- HTML files are plain text files parsed by web browsers to render visual pages
- Works alongside CSS (styling) and JavaScript (behaviour) to build modern websites

HTML Versions — A Brief History



HTML Editors

- A good HTML editor keeps your code clean, organised, and error-free
- Editors auto-close tags — reducing bugs and saving typing time
- Most modern editors include a WYSIWYG preview feature

❑ Popular Free Editors:

- Notepad / Notepad++ — lightweight plain text editing
- VS Code (Visual Studio Code) — most popular, rich extensions, IntelliSense
  - ➢ Install **Live Server** by **Ritwick Dey**
  - ➢ Open your HTML file. **Right-click** inside the HTML file and select "Open with Live Server" or click "Go Live" in the bottom-right corner.
- JSFiddle — browser-based; great for quick experiments
- HTML-Kit — dedicated HTML editor with built-in browser preview

Basic Document Structure

```
<!DOCTYPE html>          <!-- Declares HTML5 document type -->
<html lang="en">         <!-- Root element — wraps everything -->

  <head>                 <!-- Metadata (not visible on page) -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Web Page</title>   <!-- Tab title in browser -->
    <link rel="stylesheet" href="styles.css">
  </head>

  <body>                 <!-- Visible page content goes here -->
    <h1>Hello, World!</h1>
    <p>Welcome to my first web page.</p>
  </body>

</html>
```

Practical Exercise1

- Install Visual Studio Code (VS Code) on your computer and launch the application. Create a new folder named HTML_Exercise and inside it create a file called index.html. Using VS Code, write a complete HTML document and include the following content in the body section: an H1 heading titled "Viewport" followed by a paragraph explaining that “the viewport element gives the browser instructions on how to control a page's dimensions and scaling”. Add an H2 heading titled "How it works" followed by a paragraph describing how “the width=device-width setting allows the page width to match the screen width of the device.” Then add an H3 heading titled "Initial Scale" followed by a paragraph explaining that “initial-scale=1.0 sets the initial zoom level when the page is first loaded by the browser.” Save the file and run it in a web browser using either the browser directly or the Live Server extension in VS Code. Finally, submit screenshots showing the HTML code in VS Code and the webpage displayed in the browser.

Tags, Elements & Attributes

🏷️ Tags

- A **tag** is a piece of code used to build the structure of a website. It acts as a command or instruction that tells a web browser exactly how to format, organize, and display text, images, or media on the screen
  - Keywords in angle brackets: <tag>
  - Come in pairs: …
  - Opening tag: <body>
  - Closing tag: </body>
  - Empty/void tags: These tags are completely unique because they cannot contain any inner text content and do not require a closing tag.
  -
: Line break, <hr>: Horizontal rule, <img>:image
  - Browsers render tags, not display them

Tags, Elements & Attributes

📦 Elements

- People often confuse "tags" and "elements," but they are technically different. A tag is just the markup code (or), whereas an element refers to the entire bundle: the opening tag, everything written inside it, and the closing tag altogether.
  - <tagname> Content </tagname>
  - Everything from start to end tag
  - Empty elements have no content
  - Elements can be nested
  - Example: Hello World

📦 Attributes

- An **attribute** is a special modifier word placed inside an HTML opening tag that provides extra information, configurations, or settings for that element.
  - Provide extra info about elements
  - Written in the opening tag only
  - Format: name="value"
  - href, src, style, class, id …
  - HTML5 attributes are case-insensitive

Common Attribute Examples

```
<!-- Hyperlink — href attribute -->
<a href="https://www.w3schools.com">Visit W3Schools</a>

<!-- Image — src and alt attributes -->
<img src="photo.jpg" alt="A description of the image" width="300" height="200">

<!-- Inline style — style attribute -->
<p style="color: red; font-size: 18px;">This paragraph is red and large.</p>

<!-- Heading with an id -->
<h1 id="main-title">Welcome to My Website</h1>

<!-- Paragraph with a class -->
<p class="intro-text">This paragraph can be styled with CSS using .intro-text</p>
```

Practical Exercise2: Enhancing a Web Page with Attributes, Images, and Hyperlinks

- Open the HTML file created in Exercise 1 and modify it by adding a **bgcolor** attribute to the <body> tag to change the background color of the webpage. Below the **Initial Scale** paragraph, insert an image of a Bible and set an alternative text message of **"holly bible"**. The image should have a width of **100 pixels**.
- Below the image, create a paragraph that displays the text **"Hello Believers in Jesus Christ"**, with the words **"Jesus Christ"** appearing in bold. Continue the paragraph with the text **"for more information about our church visit Adventist"**, where the word **"Adventist"** functions as a hyperlink that directs users to the Adventist Church website at https://gc.adventist.org/.
- Save the file and view it in a web browser to verify that the background color, image, bold text, and hyperlink are displayed correctly.

HTML Formatting Elements(Text Formatting)

`<b>`	Bold text — visual only
`<strong>`	Important text — semantic bold
`<i>`	Italic text — visual only
`<em>`	Emphasized text — semantic italic
`<mark>`	Highlighted / marked text
`<small>`	Smaller text
`<del>`	Deleted / strikethrough text
`<ins>`	Inserted / underlined text
`<sub>`	Subscript text (e.g. H_2O)
`<sup>`	Superscript text (e.g. x^2)

- Create a single HTML page that demonstrates the use of inline text formatting tags: <b>, <i>, <mark>, <del>, <sub>, , , <small>, <ins>, and <sup>. The page should follow a standard HTML structure with <!DOCTYPE html>, <html>, <head>, and <body>. In the body, write a title for the page such as “Scientific Notation and Chemistry Notes” and format it using and to make it both bold and emphasized. Then, write a paragraph about chemistry or ICT concepts where you apply different formatting tags within the text: use <b> to highlight an important term, <i> to italicize a scientific or foreign word, <mark> to highlight a key safety instruction or important concept, and <small> to include a short note or disclaimer at the end of the paragraph.
- Include another paragraph that presents scientific expressions or formulas, making use of <sub> for subscripts (for example in chemical formulas like H_2O or CO_2) and <sup> for superscripts (for example powers like x^2 or 10^3). After that, include a sentence that demonstrates correction of information by using <del> to show incorrect text and <ins> to show the corrected version, such as correcting a chemical formula. Finally, add a concluding statement that emphasizes the importance of the topic using both and to show strong emphasis and importance. The overall content should be meaningful, well-structured, and demonstrate correct usage of all required HTML formatting tags within a coherent scientific or academic context.

HTML for Web Design

List & Table Tags

Ordered Lists • Unordered Lists • Tables

`<ol>`

Ordered Lists

`<ul>`

Unordered Lists

`<table>`

HTML Tables

What We'll Cover

01 **Unordered Lists **
Create bullet-point lists for navigation menus, feature lists, and content grouping.

02 **Ordered Lists **
Build numbered sequences for steps, rankings, instructions, and procedures.

03 **HTML Tables <table>**
Display structured data in rows and columns with headers, borders, and spans.

04 **Real-World Use Cases**
See how these tags power navigation bars, pricing tables, and data dashboards.

Unordered Lists

What is an Unordered List?

An unordered list displays items as a bulleted list without any specific order. Items are wrapped in **<li>** tags inside **<ul>**.

Bullet Style (list-style-type)

- disc Default filled circle ●
- circle Hollow circle ○
- square Filled square ■
- none No bullet (custom icons/CSS)

Common Web Design Uses

- Navigation menus
- Feature/service listings

HTML Code

```
<!-- Basic unordered list -->
<ul>
  <li>Home</li>
  <li>About Us</li>
  <li>Services</li>
  <li>Contact</li>
</ul>
<!-- Custom style -->

<ul style="list-style-type:
        circle;">
  <li>Feature 1</li>
  <li>Feature 2</li>
</ul>
```

Navigation menus:

```
<nav>
   <ul style="list-style-type: none; display:flex; gap: 20px;
padding:0; margin:0;">
       <li><a href="#">Home</a></li>
       <li><a href=“#">About</a></li>
       <li><a href="#">Contact</a></li>
       <li><a href="#">Service</a></li>
   </ul>
</nav>
```


Ordered Lists

What is an Ordered List?

An ordered list renders items with sequential numbers or letters. Uses **<ol>** as container and **<li>** for each item.

The type Attribute

type	Renders As
1 (default)	1, 2, 3, 4…
A	A, B, C, D…
a	a, b, c, d…
I	I, II, III…
i	i, ii, iii…

Common Web Design Uses

- Step-by-step tutorials
- Top-10 rankings / listicles
- Numbered FAQs and instructions

HTML Code

```
<!-- Default numbered list -->
<ol>
  <li>Install Node.js</li>
  <li>Create project folder</li>
  <li>Run npm init</li>
  <li>Write your code</li>
</ol>

<!-- Uppercase letters -->
<ol type="A">
  <li>Plan</li>
  <li>Design</li>
  <li>Develop</li>
</ol>
```

List Attributes & Nesting

start

```
<ol start="5">
```

Begin numbering from any value (e.g. 5, 10)

reversed

```
<ol reversed>
```

Count down instead of up (10, 9, 8…)

value

```
<li value="7">
```

Override numbering for a specific item

**Nested Lists — Combining <ul> and **

```
<ol>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li></ul>
  </li>
  <li>Backend</li>
</ol>
```

Renders As:

1. **Frontend**
   - HTML
   - CSS
2. **Backend**

<table>

HTML Tables

<table>
Container for the whole table

<tr>
Defines one table row

<thead>
Groups header rows

<th>
Header cell (bold + centered)

<tbody>
Groups body/data rows

<td>
Data cell (standard content)

<tfoot>
Groups footer rows

<caption>
Table title / description

Table Attributes & Code Example

Key Attributes

Attribute	Used On	Purpose
border	<table>	Add visible borders
colspan	<td>/<th>	Merge across columns
rowspan	<td>/<th>	Merge across rows
scope	<th>	Accessibility: col/row
width	<td>/<th>	Set cell width
align	<td>/<th>	Align content: left/center/right

colspan & rowspan

```
<td colspan="2">Merged Cell</td>
```

colspan merges columns, rowspan merges rows. Essential for complex data tables and pricing grids.

HTML Code

```
<table border="1">
  <caption>Student Marks
  </caption>
  <thead>
    <tr>
      <th>Name</th>
      <th>Subject</th>
      <th>Marks</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>HTML</td>
      <td>95</td>
    </tr>
  </tbody>
</table>
```

Key Takeaways

`<ul>`	Use for non-sequential items — navigation menus, feature lists. Control bullets with CSS list-style-type.
`<ol>`	Use for ordered sequences — steps, rankings, instructions. Use type, start, and reversed attributes for flexibility.
`<table>`	Use <table> with <thead>, <tbody>, <tfoot>, <tr>, <th>, <td> for structured data. colspan/rowspan merge cells.
✓ CSS	All three tags are enhanced with CSS — list-style, border-collapse, nth-child selectors for zebra striping, and Flexbox for nav bars.

Practical Exercise4: Hotel Services Webpage

- Create an HTML webpage for a local hotel with a suitable **title and main heading**. The webpage should include a **numbered list showing the following steps to book a hotel room**: visit the hotel reception or website, check available room types (include a **nested list** to show room types such as Standard Room, Deluxe Room, Family Room, and Suite), select a preferred room, provide guest details for registration, make the required payment, and receive booking confirmation and room access. Include a **bulleted list of hotel services** such as Accommodation, Restaurant, Free Wi-Fi, Laundry Services, Room Service, Conference Facilities, Airport Shuttle, and Swimming Pool. Also, **create a table with the columns** Room Type, Capacity, Price per Night, and Availability, and populate it with relevant hotel room information and data. Use appropriate HTML tags and formatting to ensure the webpage is clearly structured, readable, and professionally presented.

HTML Forms

Using HTML form elements to build real interfaces

What is an HTML Form?

Definition

An HTML form is a section of a web page that collects user input and sends it to a server or JavaScript handler for processing.

Common uses

- Login & registration
- Search boxes
- Contact & feedback
- Checkout & payments
- Settings & preferences

Key HTML tag: `<form action="/submit" method="POST"> … </form>`

Method defines how the browser sends the form data to the destination server specified in the action attribute

The **action attribute** in an HTML <form> tag defines **where the form data should be sent** when a user clicks the submit button. It specifies the URL of the server-side script, API endpoint, or destination page that will process, save, or handle the submitted information.

Anatomy of the <form> Tag

```
<form   action="/submit"
method="post"
  id="signup-form"
novalidate>
  …inputs…
</form>
```

action URL where form data is sent

method GET (retrieves data from a server) or POST (sends data to a server to create/update resources)

id Links CSS/JS and labels to the form

novalidate Disable browser's built-in validation

Core Input Types

`type="text"`

Single-line text

e.g. name, username

`type="email"`

Email with validation

e.g. user@example.com

`type="password"`

Hidden characters

e.g. login forms

`type="number"`

Numeric input

e.g. age, quantity

`type="checkbox"`

Boolean on/off

e.g. agree to terms

`type="radio"`

One of many options

e.g. gender, size

Labels & Accessibility

❌ **Without label**

```
<input type="text"
  placeholder="Enter
name">
```

Screen readers can't announce it.
Placeholder disappears when typing.

✅ **With label**

```
<label for="name">Full
name</label>
<input type="text"
  id="name"
  name="name">
```

Clicking label focuses the input.
Always readable by assistive tech.

Rule: every <input> must have a matching <label for="…"> — for= matches the input's id

More Form Elements

`<select>`

Dropdown list

```
<label
for="country">Country
</label>
<select id="country"
name="country">
  <option value="">
-- Choose --</option>
  <option
value="rw">Rwanda
</option>
  <option
value="ke">Kenya
</option>
</select>
```

`<textarea>`

Multi-line text

```
<label
for="msg">Message</la
bel>
<textarea id="msg"
name="message"
  rows="4" cols="40"
  placeholder="Type
here…">
</textarea>
```

`<button>`

Submit / reset

```
<button type="submit">
  Send message
</button>

<button type="reset">
  Clear form
</button>
```

Built-in Validation

HTML attributes that enforce rules without JavaScript:

`required`	Field must not be empty before submit
`minlength / maxlength`	Minimum/maximum character count
`min / max`	Numeric or date range limits
`pattern`	Regex pattern the value must match
`type validation`	email/url/number types auto-validate format

```
<input type="email" id="email" name="email" required
minlength="6" placeholder="you@example.com">
```

A Complete Contact Form

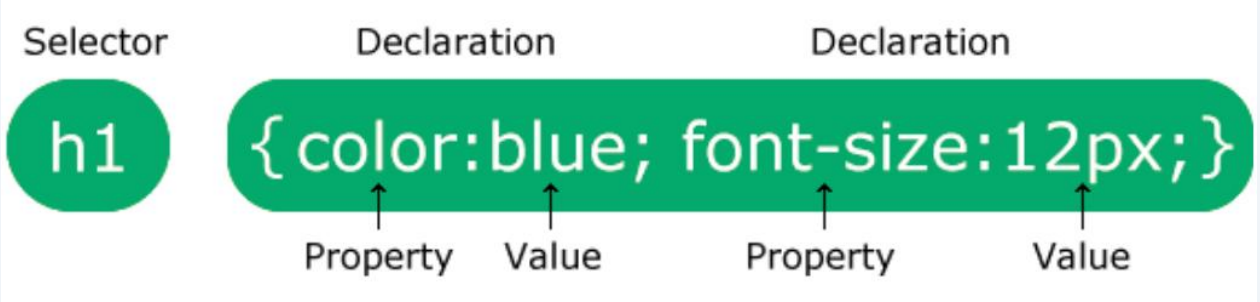
```
<form action="/contact" method="post">

  <label for="name">Full name</label>
  <input type="text" id="name" required>

  <label for="email">Email</label>
  <input type="email" id="email" required>

  <label for="msg">Message</label>
  <textarea id="msg" name="message" rows="4"></textarea>

  <button type="submit">Send</button>
</form>
```

Styling HTML Forms with CSS

A basic CSS stylesheet for a form

What is CSS?

- CSS = Cascading Style Sheets — controls the visual presentation of HTML
- With CSS you can set: colors, fonts, sizes, spacing, layout, backgrounds, animations
- Separates content (HTML) from presentation (CSS) — cleaner, more maintainable code
- Three Ways to Add CSS:
  - Inline — style attribute on individual elements: <p style="color:red;">
  - Internal — <style> block in the <head> section
  - External — separate .css file linked with <link rel="stylesheet" href="styles.css">



What this Stylesheet Does

`body`
Sets a clean font and a light gray page background

`form`
Centers the form and gives it a compact white card look

`input, select, textarea`
Makes form fields consistent in width and spacing

`button`
Styles the submit/reset buttons to match the fields

1. Styling the <body>

```
body {
    font-family: Arial;
    background: #f2f2f2;
}
```

- font-family: Arial sets a clean, readable sans-serif font for the whole page
- background: #f2f2f2 gives the page a soft light-gray background
- These two rules establish the overall look before any specific elements are styled

2. Styling the <form>

```
form {
    width: 180px;
    margin: auto;
    background: white;
    padding: 5px;
}
```

- width: 180px keeps the form small and compact (reduced size)
- margin: auto centers the form horizontally on the page
- background: white makes the form stand out as a card against the gray page
- padding: 5px adds a little breathing room inside the form's edges

3. Styling input, select, textarea

```
input, select, textarea {
    width: 95%;
    margin: 5px 0;
    padding: 2px;
}
```

- This single rule applies to ALL text fields, dropdowns, and textareas at once
- width: 95% makes every field nearly fill the form's width, keeping things aligned
- margin: 5px 0 adds vertical spacing between fields (top & bottom only)
- padding: 2px gives a small gap between the text and the field's border

4. Styling the button

```
button {
    margin-top: 4px;
    padding: 2px;
    width: 95%;
}
```

- margin-top: 4px adds space above the button, separating it from the last field
- padding: 2px keeps the button compact, matching the input fields
- width: 95% makes the button the same width as the input fields above it
- Result: a neat, uniform-looking form from top to bottom

Key Takeaways

1. Group selectors (input, select, textarea) let you style many elements with one rule
2. margin and padding control spacing inside and around elements
3. width: 95% / 100% keeps form elements visually aligned
4. A white form on a gray body creates a simple 'card' effect

Practical Exercise5: Customer registration form

- Design a customer registration form for a hotel booking website using HTML. Your form must include the following elements:
  1. **Text input field** – for Full Name
  2. **Email input field** – for Email Address
  3. **Password input field** – for Account Password
  4. **Number input field** – for Age
  5. **Checkbox inputs** – for selecting Services/Interests (e.g., Spa, Gym, Airport Pickup, Breakfast Included, Swimming Pool)
  6. **Radio buttons** – for selecting Gender (Male / Female)
  7. **Dropdown list (select)** – for selecting Home Country
  8. **Textarea** – for Feedback or Additional Requests
  9. **Submit button and Reset button**

HTML5

Semantic & Structural Elements

Writing meaningful markup: what semantic elements are, why they matter, and how to use them.

Lesson Topics: Semantics • div vs span • header, nav, main, section, article, aside, footer

What Does "Semantic" Mean?

Semantics

The study of the **meaning** of words and phrases in a language.

Semantic Elements

Elements **with a meaning** — they clearly describe their purpose to both the **browser** and the **developer**.

Think of it like labeling boxes

A box labeled **"Books"** tells you exactly what's inside, before you even open it. A box labeled **"Box 1"** tells you nothing about the contents.

```
<article>  This is a blog post...</article>
<div>  ...what is this for??</div>
```

Semantic vs. Non-Semantic Elements

Non-Semantic Elements

Tell nothing about their content. Used purely as generic containers for styling/layout.

```
<div class="box">
  Generic block-level container
</div>
<span>
  Generic inline container
</span>
```

** ** — a division/section in a document

**** — an inline container for text/elements

Semantic Elements

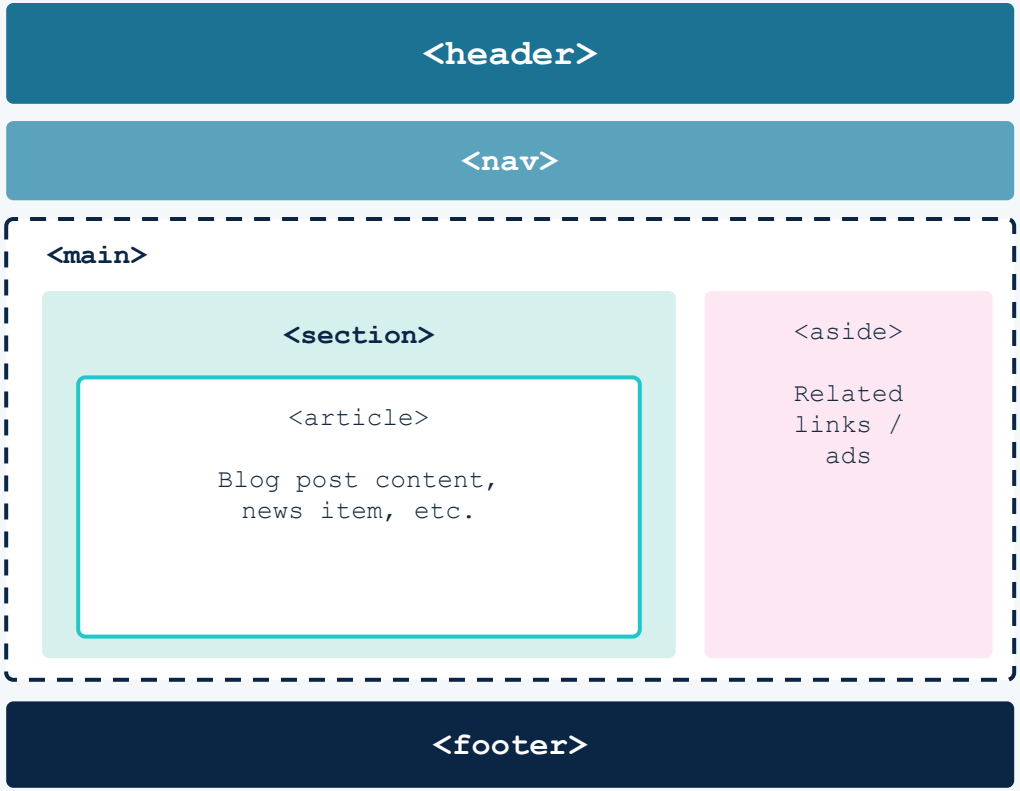
Clearly define the meaning and role of their content — for browsers, search engines, screen readers, and developers.

```
<form>...</form>
<table>...</table>
<article>...</article>
<nav>...</nav>
```

Each tag clearly states its role — a form, a table, an article, navigation.

The HTML5 Semantic Page Layout

A typical page structure built entirely from semantic elements




```
<header>
    <p>header</p>
  </header>
  <nav>
    <p>nav</p>
  </nav>
  <main>
    <section>
      <h2>section</h2>
      <article>
        <h3>article</h3>
  <p>Blog post content,<br>news item,
etc.</p>
      </article>
    </section>
    <aside>
      <h3>aside</h3>
      <p>Related links / ads</p>
    </aside>
  </main>
  <footer>
    footer
  </footer>
```

HTML5 Semantic / Structural Elements

`<header>`	Page or section header
`<main>`	Main content of document
`<article>`	Independent, self-contained content
`<figure>`	Images, diagrams with captions
`<footer>`	Page or section footer
`<span>`	Generic inline container (non-semantic)
`<nav>`	Navigation links
`<section>`	Thematic grouping of content
`<aside>`	Sidebar / tangentially related content
`<figcaption>`	Caption for a <figure>
`<div>`	Generic block container (non-semantic)

Styling a Semantic Page Layout

header • nav • main (flex) • section • article • aside • footer

1. The header

```
header {
    background: #2a7da0;
    color: #fff;
    text-align: center;
    padding: 20px;
    font-weight: bold;
}
```

- background: #2a7da0 gives the header a bold teal-blue brand color
- color: #fff makes text readable against the dark background
- text-align: center centers the title and tagline
- padding: 20px adds breathing room around the content
- font-weight: bold emphasizes the header as the page's top banner

2. The nav and nav a

```
nav {
    background: #5fa3bf;
    color: #fff;
    text-align: center;
    padding: 15px;
    font-weight: bold;
}

nav a {
    color: white;
    text-decoration: none;
    margin: 0 15px;
}
```

- ✓ nav uses a lighter blue than header, creating visual hierarchy
- ✓ nav a targets only the links inside the nav element
- ✓ text-decoration: none removes the default underline from links
- ✓ margin: 0 15px adds horizontal spacing between each link
- ✓ color: white keeps links visible against the blue background

3. The main flex container

```
main {
    display: flex;
    gap: 15px;
    padding: 15px;
    border: 2px dashed #333;
}
```

- ✓ display: flex arranges section and aside as columns in a row
- ✓ gap: 15px creates even spacing between the two columns
- ✓ padding: 15px keeps content away from the border edges
- ✓ border: 2px dashed #333 outlines the layout — handy while building/debugging

4. The section and its heading

```
section {
    flex: 3;
    background: #d6f0ea;
    padding: 15px;
}

section h2 {
    text-align: center;
    margin-bottom: 15px;
    font-size: 20px;
}
```

- flex: 3 gives section 3 parts of the row's space (wider than aside's 1 part)
- background: #d6f0ea is a soft mint to visually separate this column
- section h2 targets only headings inside section, not page-wide
- text-align: center and margin-bottom: 15px center the title and add space below it
- font-size: 20px makes the heading stand out from body text

5. The article card and its heading

```
article {
    border: 2px solid #2dd4d4;
    background: #fff;
    padding: 20px;
    border-radius: 4px;
}

article h3 {
    margin-bottom: 10px;
    font-size: 18px;
    color: #2a7da0;}
```

- ✓ border: 2px solid #2dd4d4 gives the article a bright teal outline
- ✓ background: #fff makes it stand out as a 'card' against the mint section
- ✓ border-radius: 4px softens the corners slightly
- ✓ article h3 colors the heading to match the header's blue, tying the design together
- ✓ margin-bottom: 10px separates the heading from the text below it

6. The aside sidebar

```
aside {
    flex: 1;
    background: #fbe2ec;
    padding: 15px;
}
aside h3 {
    margin-bottom: 10px;
    font-size: 18px;
    text-align: center;
}
aside ul {
    padding-left: 20px;
}
```

- flex: 1 gives aside one part of the row (narrower than section's 3 parts → 3:1 ratio)
- background: #fbe2ec is a soft pink, distinguishing it as a sidebar
- aside h3 is centered, unlike most default headings (left-aligned)
- aside ul keeps left padding so list bullets aren't flush against the edge

7. The footer

```
footer {
    background: #1c2541;
    color: #fff;
    text-align: center;
    padding: 20px;
    font-weight: bold;
}
```

- background: #1c2541 is a deep navy, the darkest color in the palette
- color: #fff ensures text contrast against the dark background
- text-align: center and padding: 20px mirror the header's styling
- font-weight: bold gives the footer the same visual weight as the header
- Together, header and footer 'bookend' the page with a consistent look

Navigation menu and Submenu

<nav>

Defines a block of major navigation links

What it's for

Wraps the primary links used to navigate the site or a section of it — main menus, tables of contents, breadcrumbs. Not every group of links needs <nav> — only major navigation blocks.

Common Use Cases

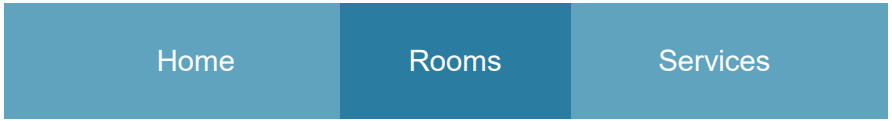
- Top main menu (Home, About, Contact)
- Sidebar table of contents
- Pagination controls (Prev / Next)

```
 <nav>
 <ul>
<li> <a href="#"> Home</a></li>
<li class="dropdown">
                <a href="#">Rooms</a>
                <ul class="submenu">
      <li><a href=“#">Standard</a></li>
      <li><a href=“#">General</a></li>
                </ul>
</li>
<li><a href="#">Services</a></li>
<li><a href="#">Bookings</a></li>
<li><a href="#">Contact Us</a></li>
</ul>
</nav>
```

Add CSS to nav

4. Navigation Bar

```css
nav ul {
  list-style-type:none;
  display:flex;
  justify-content:center;
}
nav ul li a{
      color:rgb(4, 22, 39);
         text-decoration: none;
           padding: 5px 5px;
          display: block;
       }
nav ul li a:hover {
  background:#2a7da0;
}
```



↑ *Nav preview (middle = :hover state)*

`list-style-type: none`
Removes bullet points from the

`display: flex`
Places nav items side-by-side in a row

`justify-content: center`
Centres the items horizontally

`display: block on <a>`
Makes the full cell clickable, not just the text

`:hover background`
Gives visual feedback when the user mouses over a link

5. Dropdown / Submenu

```
.submenu{
            display:none;
            position: absolute;
                }
    .dropdown:hover .submenu {
  display: block;
}
```

display: none (default)

Hides the dropdown until it is needed.

position: absolute

Removes submenu from normal flow so it floats over content below.

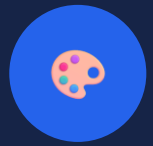
:hover → display: block

nav li { position: relative } is required on the parent so the absolute submenu anchors to it.

Practical Exercise6: Navigation menu

- Develop a **navigation menu** using HTML and CSS that contains the following items: **Home, Rooms, Services, Bookings, and Contact Us**. Under the Services menu item, create a dropdown submenu with two options: **Restaurant and Gym**. The submenu should be hidden by default and displayed when the user hovers over the Services menu item. Use appropriate HTML elements such as <nav>, <ul>, <li>, and <a> to structure the navigation menu correctly. Apply CSS styling to improve the appearance of the menu, including layout, spacing, colors, hover effects, and submenu visibility control. Ensure the use of CSS classes to properly style and manage both the main menu and the dropdown submenu.

Group A Assignment.docx
Group B Assignment.docx



- **CSS Icons**

- **Links**

- **Cursors**

Icon Libraries • Link Styling • CSS Pseudo-classes • Cursors

Eng. Eric BYIRINGIRO | Web Design| 2025–2026

CSS Icons

An icon is a small image used as a symbol to represent actions, objects, or concepts on a web page. CSS icon libraries make it easy to add scalable, styleable icons with just a single HTML tag.

Font Awesome

The most popular icon library. Free & Pro plans. Thousands of icons.

Bootstrap Icons

Open-source icon set optimized for Bootstrap. Over 1,800 icons.

Google Icons

Material Design icons from Google. Clean and minimal style.

Font Awesome Icons – Setup

Step 1 – Link the CSS in your <head>

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/7.0.1/css/all.min.css" />
```

Step 2 – Add an icon with the <i> tag

```
Rooms<i class="fas fa-chevron-down" aria-hidden="true"></i>
```
→ *Right arrow / chevron icon*

```
 <i class="fas fa-users"></i>
```
→ *Users / group icon*

```
<i class="fas fa-phone" aria-hidden="true"></i>Contact us
```
→ *Phone / contact icon*

```
<a href="#"><i class="fas fa-home" aria-hidden="true"></i>Home</a>
```
→ *Home / home icon*

The **rel** attribute specifies the **relationship** between the current document and the linked resource or destination. It is primarily used on **<a>, <link>,** and **<area>** tags to provide critical context to search engines, browsers, and assistive technologies.

CSS Links

Links (<a> tags) can be styled using any CSS property. The power of CSS is that links can look different based on their current state.

The 4 Link States (Pseudo-classes)

`a:link`

Unvisited Link

A normal link the user has never clicked

`a:visited`

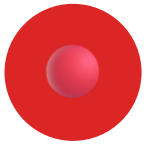
Visited Link

A link the user has already visited

`a:hover`

Hovered Link

Active when the mouse moves over the link



`a:active`

Active Link

The moment the user clicks the link

CSS Links – Styling Each State


```
nav ul li a:visited {
        color: white;
    }


    nav ul li a:focus-visible {
        outline: 2px solid #26f985;
        outline-offset: -2px;
    }

    nav ul li a:active {
        color: #130b5f;
    }
```

Key Rules to Remember

1. a:hover MUST come after a:link and a:visited
2. a:active MUST come after a:hover
3. Order matters — follow LVHA rule
4. Links inherit styles unless you override them

🧠 Mnemonic: LoVe HAte (Link > Visited > Hover > Active)

Styling awesome icons

```
.dropdown > a i.fa-angle-down {
  transition: transform 0.2s ease;
}

.dropdown:hover > a i.fa-angle-down,
.dropdown:focus-within > a i.fa-angle-down {
  transform: rotate(180deg);
}
```

CSS Cursors

The CSS cursor property specifies the mouse cursor to be displayed when pointing over an element. This is especially useful for interactive elements like links, buttons, and form controls.

```
selector { cursor: value; }
```

e.g. nav ul li a:hover { cursor: crosshair; }

cursor: default	cursor: auto	cursor: crosshair	cursor: pointer
cursor: move	cursor: text	cursor: wait	cursor: help
cursor: n-resize	cursor: s-resize	cursor: e-resize	cursor: w-resize
cursor: ne-resize	cursor: nw-resize	cursor: progress	cursor: not-allowed

Practice Activity

Task: Create a simple webpage that demonstrates CSS icons, styled links, and custom cursors.

1

Add Font Awesome Icons

- Link Font Awesome via CDN
- Display 3 different icons: fa-home, fa-envelope, fa-phone

2

Style Link States

- a:link → blue, no underline
- a:visited → purple
- a:hover → red, bold
- a:active → orange

3

Add Cursor Effects

- Set cursor: pointer on all links
- Set cursor: not-allowed on disabled buttons

Submit your HTML file to: ericbyiringiro25@gmail.com with subject: CSS Practice

Lesson Summary

Key takeaways from this lesson:

CSS Icons

Use libraries like Font Awesome, Bootstrap Icons, or Google Material Icons to add scalable icons via <i> or tags.

CSS Links

Style links with pseudo-classes: a:link, a:visited, a:hover, a:active. Always follow the LVHA order.

CSS Cursors

The cursor property changes the mouse pointer. Use cursor: pointer for links, cursor: not-allowed for disabled elements.

WEB DESIGN

JavaScript

Eng. Eric BYIRINGIRO · Instructor

What Can JavaScript Do?

- JavaScript is the programming language of the web — it runs inside the browser alongside HTML and CSS.
- It can calculate, manipulate, and validate data entered by a user.
- It can change HTML content, attributes, and styles after the page has loaded.
- It can react to events: clicks, key presses, page loads, form submissions.

The Web Trio

HTML = structure

CSS = presentation

JavaScript = behaviour

Together they build every interactive page you use.

Changing HTML With JavaScript

- **getElementById()** finds an element; **innerHTML** changes what's inside it.

```
<h2>What Can JavaScript Do?</h2>
<p id="demo">JavaScript can change HTML content.</p>
<button onclick=
  "document.getElementById('demo').innerHTML =
   'Hello JavaScript!'">
  Click Me!
</button>
```

JS

Clicking the button rewrites the paragraph text instantly — no page reload.

Where JavaScript Lives

- Code is written between <script> and </script> tags, placed in <head> or <body>.
- Scripts in <body> run as the page loads and can appear anywhere content is needed.
- External scripts keep HTML clean: <script src="myScript.js"></script>.
- One external file can be reused across many HTML pages.

Best Practice

Place scripts just before </body> or use external .js files. This keeps pages fast and code organised.

SYNTAX & OUTPUT

Ways to Display Output

JavaScript has no built-in print object — output goes through one of these.

innerHTML

- Writes into an HTML element
- document.getElementById(id).innerHTML = ...
- Most common in web pages

console.log()

- Writes to the browser console
- Used for debugging
- Never seen by end users

window.alert()

- Pops up an alert box
- window keyword is optional
- Blocks the page until closed

What Can JavaScript Do?

```
<body>
<h3>What Can Javascript do</h3>
<p id="demo">JavaScript can change HTML content</p>
<button onclick="document.getElementById('demo').innerHTML='<img
src=\'holly.jpg\'>'">Click Me</button>
<script>
  // This message will print to the console when the page loads
  console.log("The HTML page has loaded successfully.");
  // Checking the value of a variable
  let username = "Alex";
  console.log(username);
  // Displays a basic pop-up box instantly when the page loads
  window.alert("Welcome to my website!");
</script>
</body>
```

Storing Data With Variables

- Variables are containers for storing data values, declared with var, let, or const.
- An identifier is the name given to a variable — it must be unique within its scope.
- Names can contain letters, digits, underscores, and dollar signs, but cannot start with a digit.
- JavaScript identifiers are case-sensitive: age and Age are different variables.

Reserved Words

Identifiers cannot be JavaScript keywords such as let, class, return, or function.

let, const, and var

- let: value can change. const: value cannot be reassigned. var: older, function-scoped.

```
let score = 10;
score = 15;           // let can change

const rate = 0.15;
// rate = 0.2;        // Error: cannot reassign a const
 var legacyCount = 3; // still works, but avoid in new  code
```

JS

Modern JavaScript favours const by default and let when reassignment is needed.

Arithmetic & Assignment Operators

Arithmetic

- + addition
- - subtraction
- * multiplication
- / division
- % remainder
- ** exponent

Assignment

- = assign
- += add & assign
- -= subtract & assign
- *= multiply & assign
- /= divide & assign

Increment

- ++ increment by 1
- -- decrement by 1
- x++ (post) vs ++x (pre)
- Common in loop counters

Comparison & Logical Operators

Comparison

- == equal value
- === equal value & type
- != not equal
- !== strictly not equal
- > < >= <=

Logical

- && logical AND
- || logical OR
- ! logical NOT
- Used to combine conditions

Control Flow

If Conditions • Loops

Making Decisions

- if executes a block only when its condition is true.
- else runs when the if condition is false.
- else if chains additional conditions, checked in order.
- switch compares one value against many possible cases — a clean alternative to long if/else if chains.

Truthy & Falsy

0, "", null, undefined, NaN, and false are falsy. Everything else — including "0" and [] — is truthy.

Activity:

An election commission wants a simple web form where a citizen enters their age and immediately sees whether they are eligible to vote.

Condition	Output in #result
age >= 18 && age <= 65	"Eligible to vote"
age < 18 or age > 65	"Not eligible to vote"

Checking Eligibility with User Input

Reading a value from an <input> field and branching with if / else

```
<input type="number" id="ageInput" placeholder="Enter your age"><br>
<button onclick="checkAge()">Check</button>
<p id="result"></p>
<script>
  function checkAge() {
    let age = document.getElementById("ageInput").value;
    if (age >= 18 && age < 65) {
    document.getElementById("result").innerText = "Eligible to Vote";
    } else {
document.getElementById("result").innerText = "Not Eligible to Vote";
}   }
</script>
```

Asignment

Create a web page named **bookings.html** and link it to **home_page.html** through the navigation menu(Bookings). Design the page with the title **"Hotel Booking Rate Calculator"**, a text box for guests to enter the number of nights they plan to stay, a **Check Rate** button, and a paragraph or element to display the result. Using JavaScript, write a program that applies **if...else if...else** conditional statements to determine the applicable room rate. If the entered value is less than **1** or is **not a number**, display **"Please enter a valid number of nights."** If the guest stays **1–2 nights**, display **"Standard rate: $120/night."** If the guest stays **3–6 nights**, display **"Weekly discount rate: $100/night."** If the guest stays **more than 6 nights**, display **"Extended stay rate: $80/night."** Ensure that the result is displayed dynamically on the web page without reloading it.

After displaying the booking rate result on **bookings.html**, add a hyperlink with the text **"Proceed with Booking"** or **"Continue Booking"**. Configure the hyperlink so that it directs the client to **reservation.html**.

On the **reservation.html** page, design a simple hotel reservation form that allows guests to enter the following information:

1. Full Name
2. Email Address
3. Phone Number
4. Check-in Date
5. Check-out Date
6. Number of Guests
7. Room Type
8. Special Requests (optional)
9. **Book Now** button

Ensure the page has an appropriate heading, is neatly styled using CSS, and is linked correctly from **bookings.html**.

Functions, Objects & Scope

Functions • Objects • Scope

Reusable Blocks of Code

- A function is a block of code designed to perform a task, executed when it is called.
- Functions can accept parameters and optionally return a value with return.
- Function declarations are hoisted; they can be called before their definition appears.

Naming Tip

Use verbs for function names: calculateTotal(), sendEmail(), isValid(). This makes calling code read like a sentence.

Declarations, Expressions

```
function add(a, b) {          // function declaration
  return a + b;
}

const multiply = function (a, b) {// function expression
  return a * b;
};
```

JS

Functions as Values

- Functions are first-class values in JavaScript — they can be stored in variables, arrays, or object properties.
- A callback is a function passed as an argument to another function, run at a later time.
- Higher-order functions accept and/or return other functions (e.g. array .map() and .filter()).
- This pattern underlies almost every modern JavaScript API: events, promises, and array methods all lean on callbacks.

Grouping Related Data

- An object stores keyed collections of data and more complex entities as name:value pairs.
- Object properties are accessed with dot notation (obj.prop) or bracket notation (obj["prop"]).
- Methods are functions stored as object properties; inside a method, this refers to the object.
- Objects can be created with object literals {}, the new Object() constructor, or classes.

Object Literal

```
const car = {
  make: 'Toyota',
  year: 2022,
  start() { ... }
};
```

Dates & Time

Dates • Temporal

The Date Object

- Date objects let JavaScript work with dates and times, created with new Date().
- new Date() with no arguments gives the current date and time.
- Months are zero-indexed: January is 0, December is 11.
- getFullYear(), getMonth(), getDate(), and getDay() read individual components.

Known Pain Points

Date is mutable, has no real time-zone support, and its zero-indexed months trip up nearly every JS beginner.

TEMPORAL · NEW

The Temporal API

PlainDate

- Calendar date only
- No time, no time zone
- e.g. a birthday

PlainDateTime

- Date + wall-clock time
- Still no time zone
- e.g. a local appointment

ZonedDateTime

- Date + time + IANA zone
- Handles DST correctly
- Best default for real events

Temporal in Practice: To day’s date

```
<body>
<h2>Today's Date Using Temporal API</h2>
<p id="result"></p>
<!-- Temporal API Polyfill -->
<script src="https://cdn.jsdelivr.net/npm/@js-
temporal/polyfill/dist/index.umd.js"></script>
<script>
    // Get today's date using Temporal.PlainDate
    const today = Temporal.Now.plainDateISO();
    // Display today's date on the webpage
    document.getElementById("result").innerHTML =
        "<b>Today's Date:</b> " + today.toString();
</script>
</body>
```

JS

Temporal in Practice: Meeting Start time

```
<body>
<h2>Meeting Start Time Using ZonedDateTime</h2>
<p id="result"></p>
<!-- Temporal API Polyfill -->
<script src="https://cdn.jsdelivr.net/npm/@js-
temporal/polyfill/dist/index.umd.js"></script>
<script>
    // Create meeting date and time with Africa/Kigali timezone
    const meeting = Temporal.ZonedDateTime.from(
        "2026-07-07T14:00:00[Africa/Kigali]"
    );
    // Display meeting start time
    document.getElementById("result").innerHTML =
        "<b>Meeting Start Time:</b> "
        + meeting.toPlainTime().toString()
        + "<br><b>Time Zone:</b> Africa/Kigali";
</script>
</body>
```

JS

Temporal in Practice: Meeting End time

```
<body>
<h2>Meeting Schedule</h2>
<p id="result"></p>
<!-- Temporal API Polyfill -->
<script src="https://cdn.jsdelivr.net/npm/@js-temporal/polyfill/dist/index.umd.js"></script>
<script>
  // Create meeting start date and time with timezone
  const meeting = Temporal.ZonedDateTime.from(
    "2026-07-07T14:00:00[Africa/Kigali]"
  );
  // Calculate meeting ending time after 2 hours
  const endingTime = meeting.add({
    hours: 2
  });
  // Display meeting ending time and timezone
  document.getElementById("result").innerHTML =
  "<b>Meeting Ending Time:</b> "
  + endingTime.toPlainTime().toString()
  + "<br><br>" +
  "<b>Meeting Timezone:</b> "
  + meeting.timeZoneId;
</script>
</body>
</html>
```

JS

· JavaScript

Assignment: Hotel Contact Us Form Development

A hotel wants to develop a professional **Contact Us** webpage that allows customers to submit their personal information and inquiries. Create a webpage named **contact.html** and link it to the **Contact Us** menu item in home_page.html through the navigation menu. The form should include fields for the customer's **full name, email address, phone number, city, country, and age**. Apply **CSS styling** to design an attractive and user-friendly form layout. Use **JavaScript validation** to ensure that all required fields are completed correctly, the email and phone number formats are valid, and the customer's age is **above 18 years** before accepting the submission. Display appropriate validation messages to guide users when incorrect information is entered.